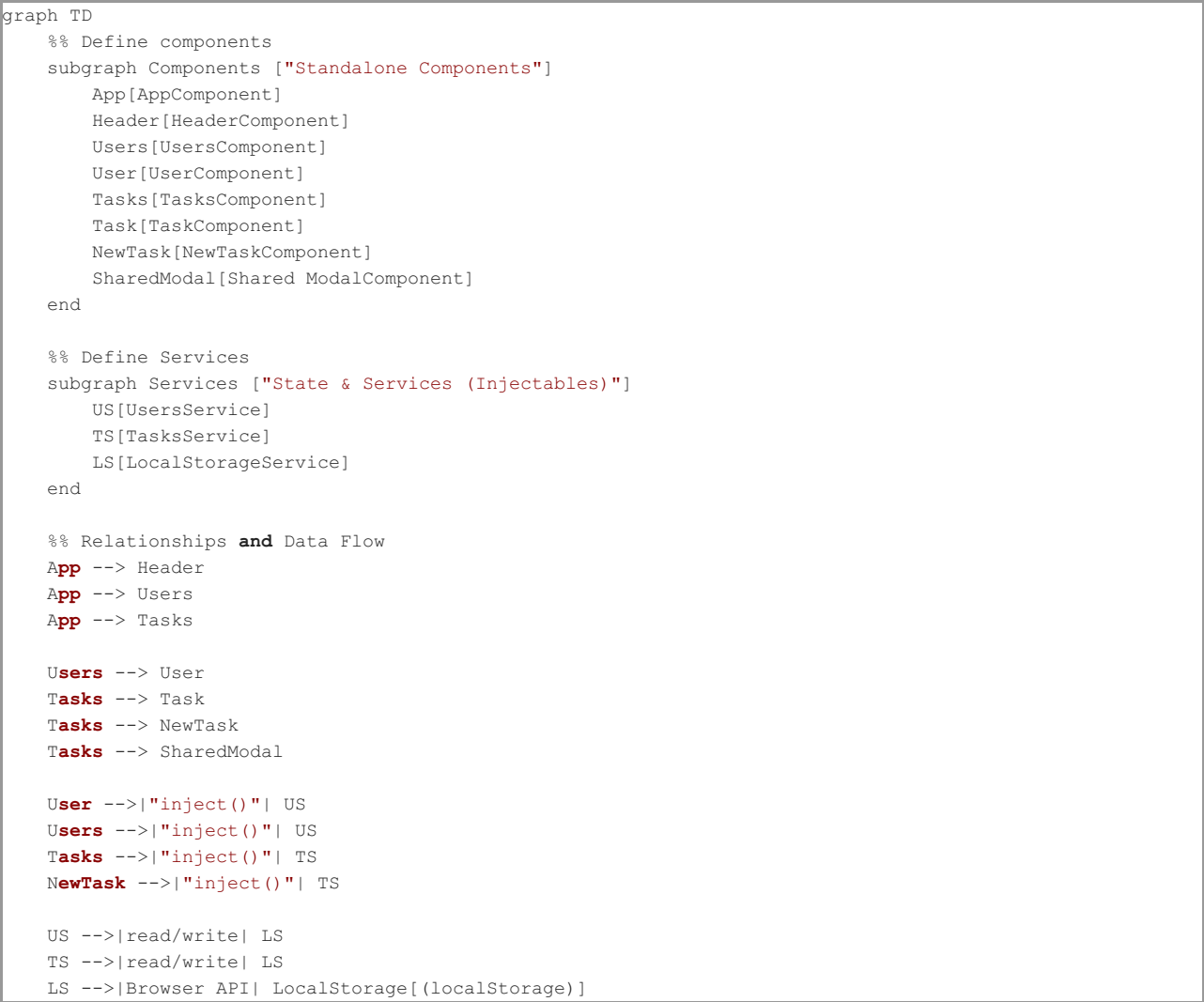


Complete Angular Project Architecture & Study Guide

🌟 Project Overview & Architecture

System Architecture and Data Flow



Component Responsibility Breakdown

Component	Responsibility	Key Concepts Used
AppComponent	Root orchestration, manages global layout and conditionally renders TasksComponent when a user is selected.	Standalone components, conditional rendering (@if)
HeaderComponent	Renders the top navigation/header. purely presentational.	Scoped CSS, static HTML
UsersComponent	Displays a list of all users, handling user selection.	@for loop, Signals, computed properties
UserComponent	Displays individual user details (avatar, name). Emits events on selection.	input.required(), output(), Host styling
TasksComponent	Manages tasks for the currently selected user. Includes filtering and spawning NewTaskComponent inside a modal.	Dependency Injection, Signals, Modals
TaskComponent	Renders a single task with completion and deletion functionality.	DatePipe, immutability
NewTaskComponent	Provides a form to create a new task and captures user input.	FormsModule, [(ngModel)], Two-Way Binding
ModalComponent	Reusable UI shell for dialogs utilizing content projection.	Content projection <ng-content>

Complete Directory Sitemap

```

src/app/
├─ app.component.css
├─ app.component.html
├─ app.component.ts
├─ app.config.ts
├─ app.routes.ts
├─ header/
│   └─ header.component.css
│   └─ header.component.html
│   └─ header.component.ts
├─ services/
│   └─ local-storage.service.ts
│   └─ tasks.service.ts
│   └─ users.service.ts
├─ shared/
│   └─ modal/
│       └─ modal.component.css
│       └─ modal.component.html
│       └─ modal.component.ts
├─ tasks/
│   └─ new-task/
│       └─ new-task.component.css
│       └─ new-task.component.html
│       └─ new-task.component.ts
│   └─ task/
│       └─ task.component.css
│       └─ task.component.html
│       └─ task.component.ts
│   └─ tasks.component.css
│   └─ tasks.component.html
│   └─ tasks.component.ts
└─ users/
    └─ user/
        └─ user.component.css
        └─ user.component.html
        └─ user.component.ts
    └─ users.component.css
    └─ users.component.html
    └─ users.component.ts

```

In-Depth Core Angular Concepts

Signals & Reactivity

Signals are a reactive primitive introduced in Angular 16+ that wrap a value and notify consumers when that value changes.

- `signal(value)`: Creates a writable signal. In `UserService`, `users = signal<User[]>([])` stores the list of users.
- `computed(() => ...)`: Creates a read-only signal derived from other signals. It is highly optimized and memoized. Useful for filtering derived state (e.g., getting tasks only for the selected user).
- `.asReadonly()`: Exposes a writable signal as read-only to external consumers, preventing unexpected mutations outside the service owning the state.
- `model()`: Used for two-way bound signals, allowing child components to read and update a value seamlessly.

Signal-Based Inputs & Outputs

- `input.required<T>()`: Defines an input property that is mandatory for the component to function. It wraps the input in a Signal, allowing reactive derivation (via `computed`) instead of relying on `ngOnChanges`.
- `output<T>()`: The modern equivalent to `EventEmitter`. E.g., `selectUser = output<string>()` allows components to safely emit strongly-typed events to their parents.

Modern Control Flow

Angular 17 introduced a declarative, built-in control flow to replace structural directives like `*ngIf` and `*ngFor`:

- `@if (condition) { ... } @else if { ... } @else { ... }`: Cleaner syntax, better type narrowing, and no need to import `NgIf`.
- `@for (item of items; track item.id)`: Dramatically improves rendering performance by requiring a `track` expression. Angular uses this track value to uniquely identify DOM nodes for updates/removals rather than recreating the whole list.

Standalone Architecture

Standalone components (`standalone: true` in the `@Component` decorator) eliminate the need for `NgModules`. Every component explicitly imports exactly what it needs (e.g., `imports: [DatePipe, FormsModule]`). This simplifies the mental model and enables better tree-shaking.

Functional Dependency Injection

Instead of injecting dependencies via constructor parameters (`constructor(private userService: UsersService)`), Angular now favors the `inject()` function:

```
private userService = inject(UsersService);
```

This reduces constructor boilerplate and makes inheritance significantly easier.

Content Projection

`SharedModalComponent` uses `<ng-content>` to allow parent components to pass custom markup into it.

- **Single-slot:** `<ng-content></ng-content>` projects everything.
- **Multi-slot:** Allows targeting specific sections (e.g., `<ng-content select="[header]"></ng-content>`) for advanced component composition.

Forms & Two-Way Binding

The `NewTaskComponent` uses template-driven forms:

- `FormsModule` must be imported in the standalone component.
- `[(ngModel)]="taskTitle"` establishes two-way binding.
- `(ngSubmit)="onSubmit()"` captures form submissions natively without page reloads.

Pipes

Pipes transform data directly in templates. The `DatePipe({ task.dueDate | date: 'fullDate' })` is used in `TaskComponent` to format ISO date strings/objects into human-readable text.

CSS Encapsulation & Scoping

Angular emulates Shadow DOM to scope CSS to its specific component.

- `:host { ... }`: Targets the component's root element itself.
- Class scoping ensures that generic classes like `.action-btn` don't bleed into other components and ruin global styles.

Persistence Service

A centralized generic service wrapper (`LocalStorageService`) simplifies interacting with the Browser's Storage API, handling `JSON.stringify` and `JSON.parse` operations transparently, preventing repetitive boilerplate and potential serialization errors in the data layers.

Bug Knowledge Base & Root Cause Analysis

Bug 1: Action Button Clipping

- **Symptom:** Complete/Delete buttons in the `TaskComponent` were overflowing their container bounds.
- **Root Cause:** A global or parent-level style applied generic `min-width: 10rem` on `<button>` elements, conflicting with the tight space of `.user-item` or `.task-card`.
- **Fix:** Overrode the width constraints or scoped the CSS more aggressively, utilizing Flexbox wrapping/shrinking (`flex-wrap, min-width: unset`).

Bug 2: UI Not Updating on User Edit

- **Symptom:** After editing a user's name, the UI failed to reflect the change despite the service holding the correct string.
- **Root Cause:** Angular Signals use `Object.is` to detect state changes. When modifying an object property in-place (`user.name = 'New'`), the object's reference memory address remains the same. The Signal evaluates `Object.is(oldRef, newRef)` as `true`, aborting the reactive UI update.
- **Fix:** Implemented strict Immutability by spreading the object into a new reference: `users.update(users => users.map(u => u.id === id ? { ...u, name: newName } : u))`.

Bug 3: Background Elements Clickable Behind Modal

- **Symptom:** When a modal was open, buttons underneath the backdrop could still be clicked.

- **Root Cause:** The `.backdrop` class lacked proper stacking context hierarchy (`z-index`) and did not intercept pointer events (`pointer-events: auto`).
- **Fix:** Assigned an absolute high `z-index: 100` to the modal and its backdrop, explicitly covering the entire viewport to intercept all click events.

Bug 4: Skipping / Off-by-One ID Generation

- **Symptom:** Newly created users received ID sequences like `u1, u3, u5...` instead of `u1, u2, u3...`
- **Root Cause:** The `newId` counter in `UserService` was being incremented both during evaluation and storage, leading to double-increments for a single creation event.
- **Fix:** Isolated the incrementing logic into a single predictable expression before assignment.

Bug 5: Date Deserialization in LocalStorage

- **Symptom:** Task dates failed to format correctly or threw errors on initialization.
- **Root Cause:** The `localStorage` API only stores strings. Calling `JSON.stringify()` on a `Date` object converts it to an ISO string. `JSON.parse()` does not magically convert it back to a `Date` object.
- **Fix:** Safely parsed the date string back into a `Date` object within the service layer during instantiation, or ensured the template specifically piped an ISO string using `DatePipe`.

Getting Started & CLI Commands

Prerequisites

- Node.js (v18+)
- Angular CLI (`npm install -g @angular/cli`)

Commands

1. Installation

```
npm install
```

Installs all dependencies mapped in `package.json`.

2. Development Server

```
ng serve
```

Compiles the application and hosts it at `http://localhost:4200/`. The app will automatically reload if you change any of the source files.

3. Build for Production

```
ng build
```

Builds the project into the `dist/` directory. Uses AOT (Ahead of Time) compilation and optimizes bundles for production deployment.

4. Running Unit Tests

```
ng test
```

Executes unit tests via `Karma/Jasmine` to ensure component and service integrity.